

Dockerized Workout MERN Application – Project Documentation

1. Project Overview

This project is a Dockerized MERN (MongoDB, Express, React, Node.js) application. The purpose is to containerize all services (frontend, backend, and database) to ensure consistent development and deployment using Docker and Docker Compose.

2. Technologies Used

- Node.js (v20)
 - React (Vite / Development Server)
 - MongoDB
 - Docker
 - Docker Compose
-

3. Architecture Overview

The application consists of three services:

- Backend → Node.js API (port 3000)
- Frontend → React app (port 5173)
- Database → MongoDB (port 27017)

All services communicate through a custom Docker bridge network:

Network Name: workout-app-network

4. Backend Docker Configuration

FROM node:20

WORKDIR /app

COPY package*.json ./

RUN npm install

COPY . .

EXPOSE 3000

CMD ["npm", "run", "start:dev"]

Explanation:

- Uses Node.js 20
 - Installs dependencies
 - Runs backend in development mode
 - Exposes API on port 3000
-

5. Frontend Docker Configuration

FROM node:20-alpine

WORKDIR /app

COPY package*.json ./

RUN npm install

COPY . .

EXPOSE 5173

CMD ["npm", "run", "dev", "--", "--host"]

Explanation:

- Lightweight Alpine Node image
 - Runs React/Vite development server
 - Exposes frontend on port 5173
-

6. Docker Compose Configuration

```
services:
  backend:
    build: ./backend
    ports:
      - "3000:3000"
    env_file:
      - ./backend/.env
    depends_on:
      - mongodb
    networks:
      - workout-app-network
```

```
frontend:
  build: ./frontend
  ports:
    - "5173:5173"
  networks:
    - workout-app-network
```

```
mongodb:
  image: mongo
  ports:
    - "27017:27017"
  volumes:
    - mongo_data:/data/db
  networks:
    - workout-app-network
```

```
volumes:
  mongo_data:
```

```
networks:
  workout-app-network:
    driver: bridge
```

7. Networking

All containers communicate using a shared bridge network:

- Network Name: workout-app-network

- Driver: bridge
- Purpose: Enables inter-service communication using container names

```
dibin@DESKTOP-JLR6PMC:/mnt/c/Users/NEW/Desktop/project/DOCKER/workout-app$ docker network ls
NETWORK ID          NAME                                DRIVER              SCOPE
f0b0afa1172b        bridge                             bridge              local
4f6f44f2b4df        host                               host                local
7b9bfef8bec3        none                               null                local
906cf3d66414        workout-app-mern_zenu-network      bridge              local
607f6e99518f        workout-app-network                bridge              local
dibin@DESKTOP-JLR6PMC:/mnt/c/Users/NEW/Desktop/project/DOCKER/workout-app$
```

8. Persistent Storage

MongoDB uses Docker volume:

- Volume: mongo_data
- Purpose: Keeps database data persistent across restarts

9. Deployment Commands

Start project:

`docker compose up --build`

Stop project:

`docker compose down`

☐	Name	Container ID	Image	Port(s)	CPU (%)	Memory	Actions
☐	workout-app-mern	-	-	-	1.54%	542.03M	  
☐	mongodb-1	69d7e3470321	mongo	-	0.66%	98.03M	  
☐	frontend-1	a35666fc76b1	workout-app	-	0.23%	135.5M	  
☐	backend-1	209797158bca	workout-app	3000:3000	0.65%	308.5M	  

```

dibin@DESKTOP-3LR6PMC: /mnt/c/Users/NEW/Desktop/project/DOCKER/workout-app-mern$ docker compose up --build
[+] Building 62.3s (24/24) FINISHED
=> [internal] load local bake definitions                                0.0s
=> => reading from stdin 1.12kB                                         0.0s
=> [frontend internal] load build definition from Dockerfile            0.1s
=> => transferring dockerfile: 190B                                       0.1s
=> [backend internal] load build definition from Dockerfile            0.1s
=> => transferring dockerfile: 173B                                       0.1s
=> [frontend internal] load metadata for docker.io/library/node:20-alpine 1.4s
=> [backend internal] load metadata for docker.io/library/node:20     1.4s
=> [auth] library/node:pull token for registry-1.docker.io            0.0s
=> [frontend internal] load .dockerignore                               0.1s
=> => transferring context: 2B                                             0.1s
=> [backend internal] load .dockerignore                               0.1s
=> => transferring context: 58B                                           0.1s
=> [frontend 1/5] FROM docker.io/library/node:20-alpine@sha256:fb4cd12c85ee03686f6af5362a0b0d56d50c58a04632e6c0fb8363f609372293 0.0s
=> => resolve docker.io/library/node:20-alpine@sha256:fb4cd12c85ee03686f6af5362a0b0d56d50c58a04632e6c0fb8363f609372293 0.0s
=> [frontend internal] load build context                               0.2s
=> => transferring context: 188.15kB                                       0.2s
=> [backend internal] load build context                               0.2s
=> => transferring context: 407.55kB                                       0.1s
=> [backend 1/5] FROM docker.io/library/node:20@sha256:8f693eaa7e0a8e71560c9a82b55fd54c2ae920a2ba5d2cde28bac7d1c01c9ba5 0.0s
=> => resolve docker.io/library/node:20@sha256:8f693eaa7e0a8e71560c9a82b55fd54c2ae920a2ba5d2cde28bac7d1c01c9ba5 0.0s
=> CACHED [backend 2/5] WORKDIR /app                                    0.0s
=> [backend 3/5] COPY package*.json ./                                  0.1s
=> CACHED [frontend 2/5] WORKDIR /app                                    0.0s
=> [frontend 3/5] COPY package*.json ./                                  0.1s
=> [backend 4/5] RUN npm install                                         33.1s
=> [frontend 4/5] RUN npm install                                        24.1s
=> [frontend 5/5] COPY . .                                              0.4s
=> [frontend] exporting to image                                         21.4s
=> => exporting layers                                                    7.4s
=> => exporting manifest sha256:672ca1df096d2c2db6517d6fd217b9eabb16ba578773130e28eb02bbfb4e2b 0.0s
=> => exporting config sha256:9d57538ac840c8e90134732263de9df3bc717e06f15ae7049a280233e0d71b1 0.0s
=> => exporting attestation manifest sha256:5701bb2fd36078e76f212a1c14d29521323bd2d4f81f871eaf152734022a55bd 0.0s
=> => exporting manifest list sha256:9cb320c4f99f6e6a604aa05959368e3acf0b52cd16974c328199c2a851b779ec 0.0s
=> => naming to docker.io/library/workout-app-mern-frontend:latest      0.0s
=> => unpacking to docker.io/library/workout-app-mern-frontend:latest    13.0s
=> [backend 5/5] COPY . .                                              0.6s
=> [backend] exporting to image                                         25.9s
=> => exporting layers                                                    14.3s
=> => exporting manifest sha256:18a127752178501998bf29632780aaa621d8da9c3fc416ccc67dc410f8798ce3 0.0s
=> => exporting config sha256:84c99508d1cf068966dea18f2d06d341deb48a74fela80d603b640593e64e3ea0 0.0s
=> => exporting attestation manifest sha256:3540441b630ce5e2250588199f7fe15986963835c4f177f9ecb3b9ae51d5daef 0.0s
=> => exporting manifest list sha256:3716aa6cd5299e00b572b78b59dddaa532d9fb36e2407dbcc43af84b8beb652 0.0s
=> => naming to docker.io/library/workout-app-mern-backend:latest        0.0s
=> => unpacking to docker.io/library/workout-app-mern-backend:latest     11.4s
=> [frontend] resolving provenance for metadata file                    0.0s
=> [backend] resolving provenance for metadata file                    0.0s
[+] up 7/7
✔ Image workout-app-mern-backend Built 62.5s
✔ Image workout-app-mern-frontend Built 62.5s
✔ Network workout-app-mern_zenn-network Created 0.1s
✔ Volume workout-app-mern_mongo_data Created 0.0s
✔ Container workout-app-mern-frontend-1 Created 1.7s
✔ Container workout-app-mern-mongodb-1 Created 1.7s

```

10. Conclusion

This project demonstrates a complete Dockerized MERN stack setup with isolated services, persistent storage, and a shared network (workout-app-network) ensuring portability and scalability.